

Transmissão Confiável de Dados via Rádio: Implementação de Controlo de Erro e Fluxo com ESP32

Mateus de Souza Arruda , Matheus Henrique Moreira Louro

1

Resumo. *Este projeto implementa um sistema de comunicação sem fio entre duas placas ESP32 utilizando módulos RF de 433 MHz. O objetivo é transmitir mensagens de texto de forma confiável. Para isso, os dados trafegam exclusivamente pelo canal de radiofrequência usando uma estrutura customizada de quadros da camada de enlace. O sistema utiliza o algoritmo CRC16-CCITT para detecção de erros e o protocolo Selective Repeat ARQ (janela deslizante) para controle de fluxo. Os testes de bancada demonstraram que o sistema detecta pacotes corrompidos, retransmite os fragmentos necessários e remonta os dados no receptor, exibindo o resultado em um display LCD via interface I2C.*

1. Introdução

A transmissão de dados via rádio utilizando módulos analógicos de 433 MHz apresenta um desafio prático significativo devido à alta suscetibilidade a ruídos e interferências eletromagnéticas no meio sem fio. Para que a comunicação seja viável e os dados não cheguem corrompidos, é necessário aplicar mecanismos de detecção e correção de erros.

O objetivo deste trabalho é implementar na prática os protocolos da camada de enlace para estabelecer um canal confiável entre dois microcontroladores. O sistema desenvolvido transmite textos e renderiza imagens utilizando os módulos RF como único meio de comunicação.

Para resolver o problema do ruído, o projeto foca em construir o empacotamento (*framing*) e a redundância do zero. Para isso, utiliza-se o campo *Frame Check Sequence* (FCS) acoplado a um protocolo de controle de fluxo de janela deslizante para gerenciar perdas de pacotes, identificar fragmentos e solicitar retransmissões automáticas apenas quando necessário.

2. Fundamentação Teórica

Esta seção apresenta os conceitos teóricos essenciais para o desenvolvimento de um sistema de comunicação confiável. O texto foca nas estratégias adotadas pela camada de enlace para mitigar a instabilidade das transmissões sem fio, com destaque para as técnicas matemáticas de detecção de erros detalhadas pela literatura clássica de redes de computadores [Forouzan 2008].

2.1. Camada de Enlace e Enquadramento

Na arquitetura de redes, a camada física lida com a transmissão bruta de bits através do meio de comunicação. Meios sem fio baseados em radiofrequência são inerentemente ruidosos, o que torna os sinais altamente vulneráveis a interferências e corrupção durante a

propagação. A camada de enlace atua justamente para contornar essa limitação física, assumindo a responsabilidade de entregar um link de comunicação íntegro para as camadas superiores [Forouzan 2008].

O primeiro passo para garantir essa integridade é o processo de enquadramento. Essa técnica agrupa o fluxo contínuo de bits em unidades gerenciáveis chamadas quadros. Para que o dispositivo receptor consiga processar a mensagem de forma correta, o transmissor precisa adicionar informações de controle aos dados úteis. O cabeçalho de um quadro inclui marcadores delimitadores de início e fim, identificadores de sequência para o controle de fluxo e campos dedicados exclusivamente à validação de integridade [Forouzan 2008].

A presença desses campos de validação introduz o conceito de redundância, que é fundamental para a detecção de erros. A redundância dita que o transmissor deve calcular bits suplementares com base na própria carga útil da mensagem e anexá-los ao final do quadro, formando um campo estrutural denominado *Frame Check Sequence* ou FCS. Ao receber a transmissão, o destino executa o mesmo algoritmo matemático e verifica se os dados sofreram alterações no trajeto. Para gerar essa redundância de forma eficaz, a literatura destaca dois métodos principais: o *Checksum* e o *Cyclic Redundancy Check*, explorados a seguir [Forouzan 2008].

2.2. Cálculo do Frame Check Sequence (FCS) via Checksum

Um dos métodos do *Checksum* foi o algoritmo base definido para essa validação. Este método apoia-se na aritmética de complemento de um e opera agrupando os dados em unidades de tamanho fixo, comumente segmentos de 8 ou 16 bits [Forouzan 2008]. O processo de cálculo ocorre em duas fases principais: a geração na origem e a verificação no destino.

Geração no Transmissor:

1. A mensagem original é tratada como uma sequência de números inteiros e dividida em blocos de k bits.
2. Todos os blocos são somados. Se a soma exceder a capacidade de k bits (gerando um bit de *carry* ou “vai-um”), esse bit excedente é removido da esquerda e somado de volta ao bit menos significativo (LSB) do resultado. Essa operação é a base da aritmética de complemento de um.
3. O resultado final dessa soma é então invertido logicamente bit a bit, trocando os 0s por 1s e vice-versa. Esse valor invertido é o *checksum*.
4. O *checksum* é anexado ao final do quadro original na forma do campo FCS e transmitido ao meio sem fio.

Exemplo Teórico de Cálculo:

Para fins de demonstração didática, considere que o transmissor precisa enviar uma carga útil muito curta composta por dois blocos de 8 bits com os valores 10001111 e 10000010. O cálculo segue os seguintes passos:

• Soma binária dos blocos:

$$10001111 + 10000010 = 100010001$$

Notamos que o resultado gerou um 9º bit (estouro de capacidade).

- **Adição do *carry* (Complemento de um):** O bit excedente da extrema esquerda é somado ao resto do valor:

$$00010001 + 1 = 00010010$$

- **Inversão (Geração do FCS):** Complementando o resultado bit a bit, obtemos o *checksum* final:

$$\sim 00010010 = 11101101$$

Verificação no Receptor:

Quando a interface de rede receptora obtém o quadro, ela não recalcula o FCS para compará-lo cegamente. Em vez disso, o método do *checksum* define que o receptor deve somar todos os dados recebidos, **incluindo** o FCS.

Utilizando o mesmo exemplo, suponha que os dados e o FCS chegaram intactos:

- **Nova soma binária:** Soma-se os dois blocos de dados recebidos e o *checksum*: A soma dos dados (já resolvida no passo anterior com o *carry*) resultou em 00010010. Somando isso com o FCS recebido (11101101):

$$00010010 + 11101101 = 11111111$$

- **Validação:** O receptor então inverte a soma total para sua validação final:

$$\sim 11111111 = 00000000$$

Se o resultado da inversão for estritamente composto por zeros, significa que, do ponto de vista algorítmico, não houve alteração nos bits durante a transmissão, e o quadro é repassado como válido para o sistema [Forouzan 2008]. Caso contrário, qualquer bit corrompido por ruído no meio sem fio geraria um resultado diferente de zero na validação, sinalizando a quebra de integridade para que a camada de enlace descarte o pacote e aguarde a retransmissão.

2.3. Cálculo do FCS via Cyclic Redundancy Check (CRC)

Embora o *Checksum* seja um método de fácil implementação computacional, ele apresenta limitações matemáticas na detecção de erros complexos, como a inversão simultânea de múltiplos bits na mesma posição de blocos diferentes. Para sistemas que exigem um grau elevado de confiabilidade, podemos utilizar o *Cyclic Redundancy Check* (CRC).

O CRC é uma técnica mais sofisticada, fundamentada na álgebra polinomial e na divisão binária de módulo 2. Em vez de somar os blocos de dados, o transmissor trata toda a carga útil da mensagem como um único e extenso número binário. Esse número é dividido por um valor fixo predeterminado, conhecido como polinômio gerador. O resto matemático dessa divisão constitui o código CRC, que é então anexado ao final do quadro como o FCS [Forouzan 2008].

O processo lógico ocorre da seguinte maneira:

1. O transmissor e o receptor são configurados previamente com o mesmo polinômio gerador de $n + 1$ bits.
2. O transmissor anexa n bits com valor zero ao final da mensagem original.

3. Realiza-se a divisão binária módulo 2 (que equivale à operação lógica XOR) dessa mensagem estendida pelo polinômio gerador.
4. O resto resultante dessa divisão, que terá exatamente n bits de comprimento, substitui os zeros adicionados no passo 2, formando o quadro final a ser transmitido.
5. No receptor, o quadro inteiro recebido é dividido pelo mesmo polinômio gerador. Se o resto da divisão for zero, atesta-se a integridade dos dados. Se houver qualquer resto diferente de zero, um erro de transmissão é detectado e o quadro é rejeitado [Forouzan 2008].

Exemplo Teórico de Cálculo Modulo 2:

Para ilustrar o mecanismo de forma simplificada, considere que o dado original a ser transmitido seja $D = 1010$ e que o polinômio gerador escolhido seja $G = 1011$ (composto por 4 bits, logo, $n = 3$).

- **Preparação:** Adicionamos 3 zeros ao dado original, formando o dividendo:

1010000

- **Divisão (XOR):** O divisor 1011 é alinhado aos bits mais significativos para iniciar a operação de ou-exclusivo.

$$\begin{array}{r} 1010000 \\ \oplus 1011 \\ \hline 0001000 \end{array}$$

Como os próximos dois bits à esquerda do resultado parcial são zeros, avançamos e alinhamos o gerador com o próximo bit de valor 1:

$$\begin{array}{r} 0001000 \\ \oplus \quad 1011 \\ \hline 0000011 \end{array}$$

- **Transmissão:** O resto da divisão, 011, substitui os zeros inseridos na preparação. O quadro que efetivamente viaja pelo meio sem fio será:

1010011

- **Verificação no Receptor:** O receptor obtém a sequência 1010011 e divide pelo mesmo gerador 1011.

$$\begin{array}{r} 1010011 \\ \oplus 1011 \\ \hline 0001011 \\ \oplus \quad 1011 \\ \hline 0000000 \end{array}$$

O resto final sendo 000 prova matematicamente ao receptor que a mensagem chegou intacta [Forouzan 2008].

Para sistemas práticos e sujeitos a altos índices de ruído, como os módulos RF de 433 MHz utilizados neste projeto, empregam-se polinômios de maior ordem. Como será demonstrado na implementação, utilizou-se o padrão internacional CRC16-CCITT, cujo polinômio gerador produz um FCS de 16 bits, oferecendo uma barreira eficiente contra erros em rajada.

2.4. Controle de Fluxo e de Erro via Protocolos ARQ

Apenas detectar a presença de um erro algorítmico não é suficiente para viabilizar a comunicação. O sistema necessita de um mecanismo capaz de recuperar o dado perdido ou descartado. Na camada de enlace, o controle de erros atua em conjunto com o controle de fluxo por meio de protocolos de Requisição de Repetição Automática (ARQ) [Forouzan 2008].

Esses protocolos garantem a entrega confiável utilizando confirmações de recebimento, denominadas ACK, atreladas a temporizadores. A teoria de redes define três abordagens principais para implementar essa lógica de retransmissão [Forouzan 2008]:

1. *Stop-and-Wait ARQ*

Este é o mecanismo de controle mais elementar. O seu funcionamento estabelece que o transmissor deve enviar um único quadro de dados e pausar sua execução, iniciando um temporizador. O dispositivo remetente permanece ocioso aguardando o retorno de um quadro ACK correspondente. Se a confirmação chegar intacta dentro do prazo, o próximo pacote da fila é transmitido.

Caso ocorra perda de pacotes no canal físico ou o pacote chegue corrompido, o receptor o ignora silenciosamente. O esgotamento do tempo limite no remetente aciona automaticamente a retransmissão do mesmo quadro. Para evitar ambiguidades e a aceitação de pacotes duplicados, os quadros recebem numeração de sequência. Na sua forma mais básica, essa numeração alterna estritamente entre zero e um, caracterizando o Protocolo do Bit Alternante [Forouzan 2008].

2. *Go-Back-N ARQ*

O método *Stop-and-Wait ARQ* subutiliza a capacidade do canal, pois a via de comunicação fica vazia enquanto o transmissor aguarda a confirmação. O *Go-Back-N* resolve esse gargalo introduzindo o conceito de janela deslizante. O transmissor é autorizado a enviar múltiplos quadros em sequência até preencher o tamanho máximo de sua janela, sem a necessidade de esperar pelo ACK do primeiro envio [Forouzan 2008].

No entanto, o destino aceita pacotes rigorosamente na ordem correta. Se um quadro for perdido ou corrompido por ruído, o receptor o descarta e passa a ignorar todos os quadros subsequentes, mesmo que cheguem intactos. Ao perceber a quebra de sequência ou o estouro do tempo, o transmissor é obrigado a retroceder e retransmitir o pacote defeituoso juntamente com todos os outros que haviam sido enviados logo em seguida [Forouzan 2008].

3. *Selective Repeat ARQ*

O descarte de quadros corretos no *Go-Back-N* gera desperdício considerável de largura de banda em meios sujeitos a muitas falhas, como as conexões de radiofrequência. O *Selective Repeat* otimiza esse cenário ao retransmitir estritamente os pacotes que sofreram danos [Forouzan 2008].

Nesta abordagem avançada, tanto o transmissor quanto o receptor operam com janelas deslizantes independentes. O receptor armazena os pacotes que chegam fora de ordem em memórias temporárias, também conhecidas como *buffers*. Cada pacote enviado possui seu próprio temporizador no remetente. Caso um quadro falhe, o transmissor

reenvia exclusivamente o dado faltante. Assim que o receptor recebe o quadro que faltava, ele remonta a sequência completa e a entrega de forma ordenada às camadas superiores [Forouzan 2008].

3. Arquitetura do Sistema e Hardware

O circuito foi desenvolvido separando as funções em dois nós distintos: transmissor e receptor.

3.1. Materiais Utilizados

O hardware empregado consistiu em:

- 2 Placas de desenvolvimento ESP32;
- 1 Par de módulos RF 433 MHz (emissor e receptor);
- 1 Display LCD 20x4 com adaptador I2C (no receptor);
- Cabos *jumper* e protoboards.

A alimentação foi feita diretamente via porta USB do computador. O regulador interno do ESP32 (pino VIN 5V) alimentou o display LCD, enquanto o pino 3V3 alimentou os módulos de rádio.

3.2. Diagrama e Roteamento de Conexões

Para garantir a total reprodutibilidade do experimento, o circuito foi estruturado de maneira padronizada. A interface entre as placas ESP32 e os periféricos exigiu o mapeamento das portas de entrada e saída de propósito geral (GPIO), respeitando as especificações e limitações arquiteturais do microcontrolador [Santos and Santos 2024], bem como o compartilhamento adequado das linhas de alimentação. O projeto elétrico considerou as necessidades específicas de roteamento tanto para a modulação de sinais de radiofrequência [McCauley 2024] quanto para a comunicação do display LCD via barramento I2C. A disposição física do protótipo desenvolvido pode ser observada na Figura 1.

O roteamento dos sinais foi estabelecido da seguinte maneira:

Transmissor (TX):

- Módulo RF (pino DATA) → conectado à porta GPIO 25 do ESP32.
- Módulo RF (pino VCC) → conectado ao pino de alimentação 3V3 do ESP32.
- Módulo RF (pino GND) → conectado ao pino GND do ESP32.

No nó receptor (RX):

- Módulo RF (pino DATA) → conectado à porta GPIO 26 do ESP32.
- Módulo RF (pino VCC) → conectado ao pino de alimentação 3V3 do ESP32.
- Módulo RF (pino GND) → conectado ao pino GND do ESP32.

Display LCD no nó receptor (barramento I2C):

- Pino SDA (dados) → conectado à porta GPIO 21 do ESP32.
- Pino SCL (*clock*) → conectado à porta GPIO 22 do ESP32.
- Pino VCC → conectado ao pino de alimentação 5V (VIN) do ESP32.
- Pino GND → conectado ao pino GND do ESP32.

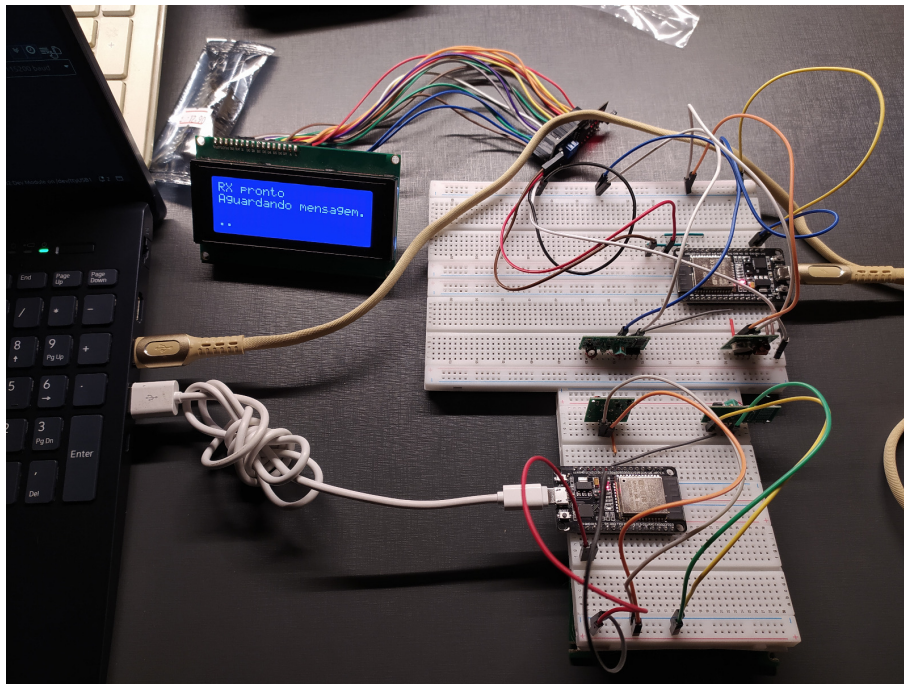


Figura 1. Montagem do circuito.

4. Implementação do Protocolo de Comunicação

O software foi escrito em C++ (framework Arduino). A camada física do rádio utilizou a biblioteca RH_ASK operando a 500 bps. O código-fonte está disponível no GitHub ¹.

4.1. Estrutura do Quadro e Controle de Erro

Os pacotes foram estruturados utilizando a `struct DecodedFrame`. Para evitar instabilidade no sinal RF, o *payload* máximo foi definido em 24 bytes. O quadro possui um byte mágico delimitador (0xA5), tipo do pacote (DATA, ACK ou END), número de sequência e comprimento, seguidos do *payload* e 2 bytes de FCS.

A verificação de erros é definida pela diretiva `USE_CRC16`. Na versão final de testes, utilizou-se a função `crc16_ccitt()`. Na recepção, a rotina `decodeFrame()` recalcula o CRC. Se o valor não bater com o recebido no quadro, o pacote é classificado com `FRAME_BAD_FCS` e ignorado.

4.2. Lógica do Selective Repeat ARQ

O sistema emprega o controle de fluxo *Selective Repeat*. O nó transmissor (`sendBufferReliable()`) mantém uma janela deslizante de 4 pacotes (`WINDOW_SIZE = 4`) e rastreia o *timeout* (2500 ms) de cada envio de forma isolada. Uma rotina de escuta não-bloqueante verifica a chegada dos ACKs. Apenas os quadros que estouram o tempo sem confirmação são reenviados.

No nó receptor, a função `processValidFrame()` usa um vetor booleano (`chunkReceived`) para anotar os fragmentos válidos, armazenando-os dinamicamente

¹<https://github.com/mateus-s-a/ESP32-Atividade-III.git>

por meio do deslocamento na memória (*offset*). Quando o sinal de término (END) chega, o receptor cruza os dados do vetor e só repassa a mensagem para as camadas superiores se todos os blocos estiverem presentes.

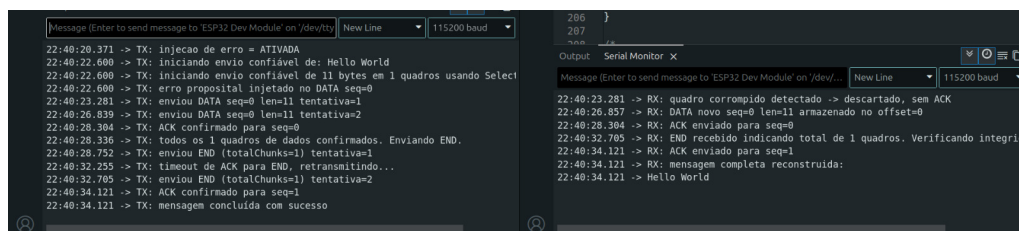
Para renderização, mensagens de texto são formatadas em quebras de linha para as limitações do LCD 20x4, enquanto cabeçalhos específicos (como [IMG:ASCII] ou [IMG:GLYPH]) desviam o código para rotinas de desenho da matriz no display.

5. Resultados e Validação

Os testes de bancada avaliaram tanto a recuperação de falhas quanto o fluxo de envio contínuo das mensagens através do monitor serial do Arduino IDE e do LCD.

5.1. Recuperação de Erros

Para testar o ARQ e o CRC16, injetou-se um erro intencional (operação lógica XOR) no *payload* da primeira tentativa de envio de um quadro. A Figura 2 mostra a captura do log. O receptor identificou a corrupção do pacote no cálculo do FCS e bloqueou o ACK. Consequentemente, ocorreu o *timeout* no transmissor, que disparou a retransmissão pontual do fragmento e obteve sucesso na segunda tentativa.



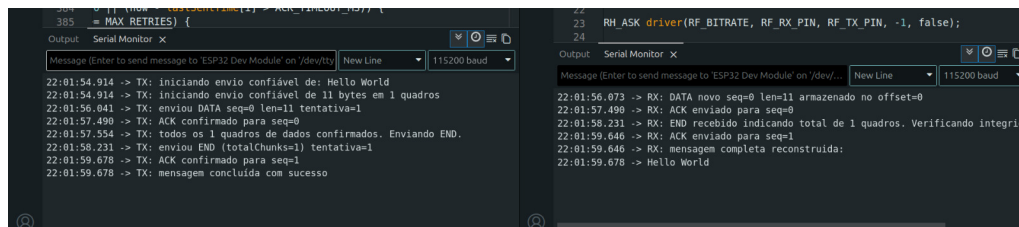
```
Message (Enter to send message to 'ESP32 Dev Module' on '/dev/tty...') New Line 115200 baud
22:40:20.371 -> TX: injeção de erro = ATIVADA
22:40:22.600 -> TX: iniciando envio confiável de: Hello World
22:40:22.600 -> TX: iniciando envio confiável de: 11 bytes em 1 quadros usando Select
22:40:22.600 -> TX: erro proposital injetado no DATA seq=0
22:40:23.281 -> TX: enviou DATA seq=0 len=11 tentativa=1
22:40:26.839 -> TX: enviou DATA seq=0 len=11 tentativa=2
22:40:28.304 -> TX: ACK confirmado para seq=0
22:40:28.336 -> TX: todos os 1 quadros de dados confirmados. Enviando END.
22:40:28.752 -> TX: enviou END (totalChunks=1) tentativa=1
22:40:32.255 -> TX: timeout de ACK para END, retransmitindo...
22:40:32.705 -> TX: enviou END (totalChunks=1) tentativa=2
22:40:34.121 -> TX: ACK confirmado para seq=1
22:40:34.121 -> TX: mensagem concluída com sucesso

22:40:23.281 -> RX: quadro corrompido detectado -> descartado, sem ACK
22:40:26.857 -> RX: DATA novo seq=0 len=11 armazenado no offset=0
22:40:28.304 -> RX: ACK enviado para seq=0
22:40:32.705 -> RX: END recebido indicando total de 1 quadros. Verificando integridade
22:40:34.121 -> RX: ACK enviado para seq=1
22:40:34.121 -> RX: mensagem completa reconstruída:
22:40:34.121 -> RX: Hello World
```

Figura 2. Monitor serial mostrando descarte por corrupção e retransmissão via *timeout*.

5.2. Exibição de Mensagens

O segundo ensaio testou o empacotamento da mensagem "Hello World". Por ter apenas 11 bytes, coube em um único quadro de dados. A Figura 3 apresenta o envio do quadro DATA, o retorno rápido do ACK e o envio da *flag* END fechando a conexão lógica.



```
385 = MAX_RETRIES {
22:01:54.914 -> TX: iniciando envio confiável de: Hello World
22:01:54.914 -> TX: iniciando envio confiável de: 11 bytes em 1 quadros
22:01:56.041 -> TX: enviou DATA seq=0 len=11 tentativa=1
22:01:57.490 -> TX: ACK confirmado para seq=0
22:01:57.554 -> TX: todos os 1 quadros de dados confirmados. Enviando END.
22:01:58.231 -> TX: enviou END (totalChunks=1) tentativa=1
22:01:59.678 -> TX: ACK confirmado para seq=1
22:01:59.678 -> TX: mensagem concluída com sucesso

23 RH_ASK_driver(RF_BITRATE, RF_RX_PIN, RF_TX_PIN, -1, false);
24
22:01:56.073 -> RX: DATA novo seq=0 len=11 armazenado no offset=0
22:01:57.490 -> RX: ACK enviado para seq=0
22:01:58.231 -> RX: END recebido indicando total de 1 quadros. Verificando integridade
22:01:59.646 -> RX: ACK enviado para seq=1
22:01:59.646 -> RX: mensagem completa reconstruída:
22:01:59.678 -> RX: Hello World
```

Figura 3. Log atestando o envio e confirmação limpa do texto de teste.

O nó receptor processou os dados I2C e apresentou o texto diretamente no display (Figura 4), mostrando que nenhum byte foi perdido ou alterado.

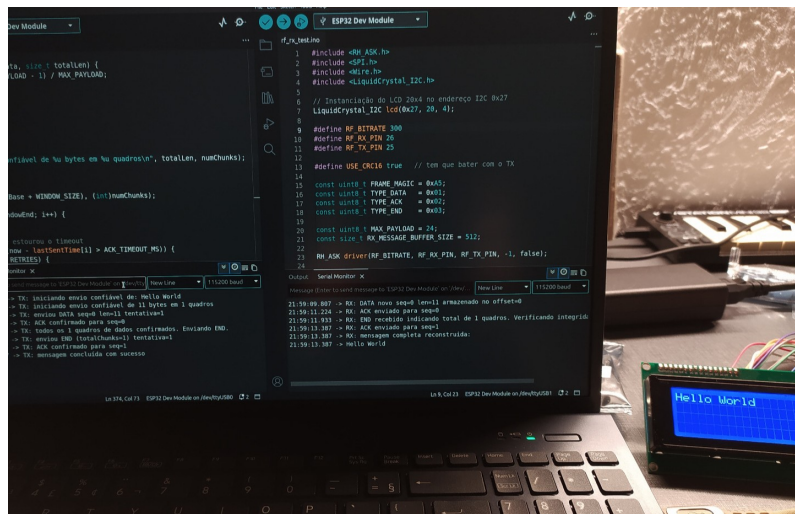


Figura 4. Display LCD do receptor exibindo a mensagem "Hello World".

6. Conclusão

A implementação prática demonstrou que a utilização de módulos RF de 433 MHz, embora simples e de baixo custo, exige obrigatoriamente uma camada de enlace robusta devido ao alto índice de ruído do meio. O uso do algoritmo CRC16-CCITT mostrou-se eficaz na detecção de quadros corrompidos, garantindo que nenhum dado inconsistente fosse processado pelo nó receptor.

Além disso, a adoção do protocolo *Selective Repeat ARQ* otimizou o uso do canal de comunicação, permitindo a retransmissão pontual de fragmentos perdidos sem a necessidade de retroceder toda a janela de transmissão, como ocorreria no método *Go-Back-N*. O sistema foi capaz de reconstruir mensagens de texto de forma íntegra, validando os conceitos de enquadramento e controle de fluxo estudados.

Como limitação observada, a taxa de 500 bps foi necessária para manter a estabilidade, o que sugere que para aplicações com maior volume de dados seria necessário o uso de módulos com modulação digital mais avançada ou a otimização dos tempos de *timeout* da biblioteca *RH_ASK*. Em suma, o projeto atingiu os requisitos técnicos ao transformar um canal físico instável em um enlace de dados confiável.

Referências

- Forouzan, B. A. (2008). *Comunicação de Dados e Redes de Computadores*. McGraw-Hill, São Paulo, 4 edition.
- McCauley, M. (2024). RadioHead Packet Radio library for embedded microprocessors. <https://docs.arduino.cc/libraries/radiohead/>. Acessado em: 15 jun. 2026.
- Santos, R. and Santos, S. (2024). ESP32 Pinout Reference: Which GPIO pins should you use? <https://randomnerdtutorials.com/esp32-pinout-reference-gpios/>. Acessado em: 15 jun. 2026.